Lab Experiment 11

Map Coloring using Constraint Satisfaction Problem (CSP)

Aim

To write a Python program to solve the Map Coloring problem using the Constraint Satisfaction Problem (CSP) approach.

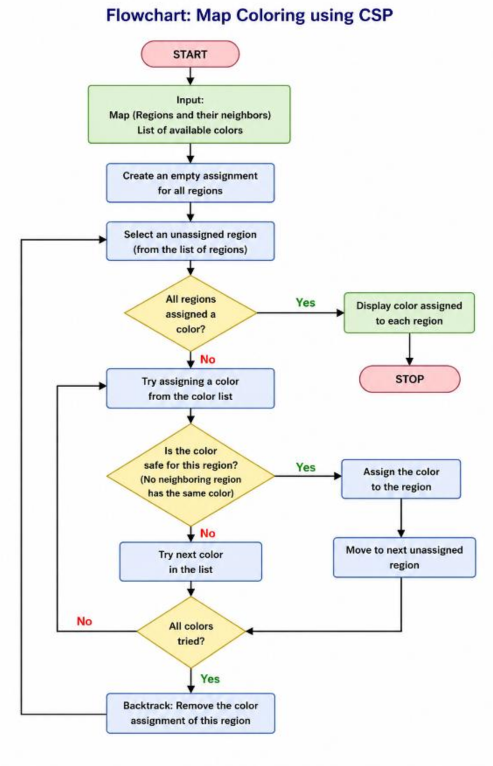
Objective

The objective of this experiment is to implement the Constraint Satisfaction Problem (CSP) technique for map coloring. It aims to assign different colors to adjacent regions without violating the coloring constraints.

Algorithm

1. Define the map as a graph with regions and their neighboring regions.
2. Assign a list of available colors.
3. Select an unassigned region.
4. Assign a color that does not conflict with neighboring regions.
5. If no valid color is available, backtrack and try another color.
6. Repeat until all regions are assigned valid colors.
7. Display the color assigned to each region.

Flowchart



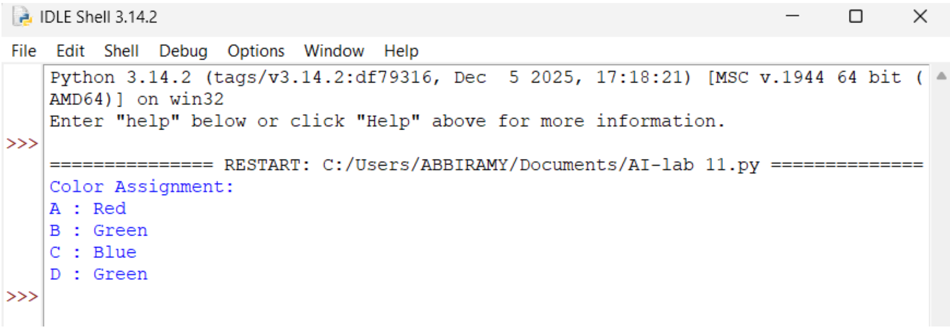
Code

```
# Map Coloring using Backtracking (CSP)


graph = {
    'A': ['B', 'C', 'D'],
    'B': ['A', 'C'],
    'C': ['A', 'B', 'D'],
    'D': ['A', 'C']
}


colors = ['Red', 'Green', 'Blue']
result = {}
```

```
def is_safe(node, color):
    for neighbor in graph[node]:
        if neighbor in result and result[neighbor] == color:
            return False
    return True

def solve(node_list, index):
    if index == len(node_list):
        return True

    node = node_list[index]

    for color in colors:
        if is_safe(node, color):
            result[node] = color

            if solve(node_list, index + 1):
                return True

            del result[node]

    return False

nodes = list(graph.keys())

if solve(nodes, 0):
```

```
    print("Color Assignment:")
    for node in result:
        print(node, ":", result[node])
else:
    print("No solution exists.")
```

Output



Result

The Python program successfully solved the Map Coloring problem using the Constraint Satisfaction Problem approach by assigning valid colors to all regions.

Conclusion

The CSP approach efficiently solves the Map Coloring problem by satisfying all constraints through backtracking. It ensures that no two adjacent regions have the same color.